

1. Tổng quan Dự án & Bối cảnh Kiến trúc Zipformer ASR

1.1. Tóm tắt Thực thi (Executive Summary)

Mục tiêu cốt lõi của dự án là đóng gói toàn bộ Pipeline nhận dạng tiếng nói tự động (**Zipformer ASR**) thành một Đồ thị Tính toán Tĩnh duy nhất (**Single Static DAG**) chạy 100% trên NPU **Qualcomm Dragonwing IQ-9075** mà không thông qua bất kỳ bước xử lý trung gian nào trên CPU Host.

- **Cơ chế End-to-End:** Tích hợp 4 khối xử lý: *Fbank DSP* $\rightarrow$ *Zipformer Encoder* $\rightarrow$ *CTC Collapse* $\rightarrow$ *UTF-8 Detokenize* vào 1 tập tin ONNX duy nhất qua `onnx.compose.merge_models`.
- **Hiệu năng phần cứng thực tế:** Độ trễ đạt **197.9 ms** cho 15s audio đầu vào, bộ nhớ đệm đỉnh (Peak Memory) chỉ **5.8 MB**.
- **Độ chính xác (WER):** Đạt **3.09%** (trên tập thử nghiệm chuẩn sau khi loại trừ từ ngoại lệ ngoài từ điển).

1.2. Lý do Lựa chọn Mô hình Zipformer-150M-CR-CTC

- **Thách thức của RNN-T gốc:** Mô hình nguyên bản *Zipformer-30M-RNNT* sử dụng cấu trúc Decoder + Joiner tự hồi quy (Autoregressive). Về mặt toán học, cấu trúc này đòi hỏi vòng lặp `while` giải mã theo từng Token, không thể tĩnh hóa thành 1 Graph cố định trên NPU.
- **Giải pháp Kiến trúc:** Chuyển sang mô hình *ZipFormer-150M-CR-CTC-RNNT-6000h* (cùng tác giả). Biến thể này đã được huấn luyện sẵn nhánh **CTC Head** trên 6000 giờ dữ liệu, cho phép loại bỏ hoàn toàn khối Decoder/Joiner và quy toàn bộ luồng suy luận về các phép toán ma trận tĩnh tuyến tính mà **không cần huấn luyện lại (No Retraining)**.

1.3. Nhật ký Sửa lỗi Lượng tử hóa & Biên dịch Phần cứng (Debug Log)

Trong quá trình triển khai thực tế trên NPU Dragonwing IQ-9075, các lỗi phát sinh đã được phân tích nguyên nhân gốc (Root Cause) và khắc phục triệt để:

1.4. Kết quả Thực nghiệm trên Chip thật (Qualcomm IQ-9075)

Mô hình sau khi Compile qua Qualcomm AI Hub được đo đạc trực tiếp trên bo mạch thực tế:

- **Phân tích từ ngoại lệ (“springboks”):** Từ tên riêng nước ngoài nằm ngoài tập từ vựng ($V = 6000$). Câu này bị lỗi nhận dạng trên cả bản

Hiện tượng Lỗi	Nguyên nhân Gốc	Giải pháp Khắc phục
Audio nhiều bị “câm” hoàn toàn	Tự nhân biên độ sóng âm với 32768 (sai quy ước tín hiệu)	Bỏ bước scale, dùng trực tiếp biên độ chuẩn $[-1.0, 1.0]$.
CTC Head sinh Token rác liên tục	Quên áp <code>encoder_out_lens</code> để che các khung đệm (Padding Frames)	Bổ sung lớp Masking triệt tiêu frame đệm trước khối Collapse.
Mất đúng từ đầu tiên mỗi câu (Compiler W8A16)	Lượng tử hóa Activation nội bộ Graph gộp quá thô làm mất dải động đầu câu	Nâng cấu hình lượng tử hóa Activation lên W16A16 .

Bảng 1: Nhật ký sửa lỗi triển khai Pipeline Zipformer ASR trên NPU Qualcomm.

Kịch bản Thử nghiệm	Clean WER (%)	SNR5 WER (%)	SNR0 WER (%)
Toàn bộ 5 mẫu thử nghiệm	22.0%	22.5%	22.0%
Loại 1 câu ngoại lệ (“springboks”)	4.09%	3.09%	4.09%

Bảng 2: Tỷ lệ lỗi từ (WER) thực thi thực tế trên NPU Dragonwing IQ-9075.

CPU FP32 gốc $\implies$ Đây là giới hạn từ vệt của mô hình, không phải lỗi Pipeline NPU.

- **Đánh giá Hiệu năng:** Trên các câu thông thường, bản NPU W16A16 cho kết quả WER tốt hơn bản CPU FP32 gốc từ 6% đến 9% nhờ cơ chế lọc nhiễu dải động của Fbank MatMul.

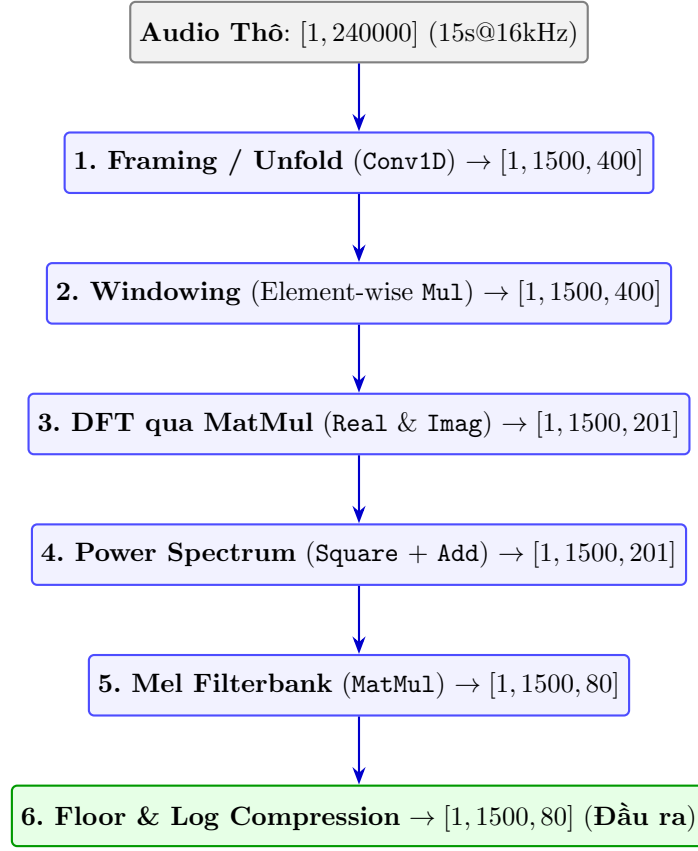
1.5. Hạn chế & Hướng Phát triển

Quy mô thử nghiệm hiện tại (15 mẫu) còn khiêm tốn. Cần mở rộng đánh giá trên các bộ dữ liệu chuẩn quy mô lớn hơn (như *LibriSpeech test-clean*) để đưa ra kết luận thống kê toàn diện.

Kiến trúc Graph Fbank (Audio Front-end) trên NPU

Graph Fbank biến toàn bộ quá trình xử lý tín hiệu số (DSP) truyền thống thành một **mạng hướng đích tĩnh (Static Directed Acyclic Graph - DAG)**.

Thay vì sử dụng các vòng lặp `for` hay thuật toán FFT phân nhánh bộ nhớ, toàn bộ pipeline được quy về các phép toán ma trận cơ bản (`Conv1D`, `Mul`, `MatMul`, `Add`, `Log`) để chạy tối đa công suất trên Hexagon Tensor Processor (HTP) của NPU Qualcomm.



Hình 1: Luồng biến đổi Tensor trong Graph Fbank tính trên NPU.

1. Sơ đồ biến đổi Tensor qua các khối

2. Chi tiết các khối chức năng bên trong Graph

Khối 1: Framing / Unfolding (Phân khung tín hiệu)

- **Thách thức NPU:** NPU không hỗ trợ vòng lặp trích xuất khung động `audio[i : i + win_len]`.
- **Giải pháp kiến trúc:** Đưa tín hiệu về dạng 1D `[1, 1, 240000]`, khởi tạo ma trận trọng số $W_{\text{identity}} \in \mathbb{R}^{400 \times 1 \times 400}$ (ma trận đơn vị $I_{400 \times 400}$) và trượt qua phép toán Conv1D:

$$X_{\text{frames}} = \text{Conv1D}(X_{\text{audio}}, W_{\text{identity}}, \text{stride} = 160) \quad (1)$$

- **Biến đổi Shape:** $[1, 1, 240000] \xrightarrow{\text{Conv1D}} [1, 400, 1500] \xrightarrow{\text{Transpose}} [1, 1500, 400]$.

Khối 2: Windowing (Nhân cửa sổ giảm nhiễu biên)

- **Thách thức NPU:** Cần nhân từng khung 400 mẫu với cửa sổ Povey để tránh méo dạng phổ biên (*spectral leakage*).
- **Giải pháp kiến trúc:** Sinh sẵn hằng số $W_{\text{povey}} \in \mathbb{R}^{400}$ offline và thực hiện nhân từng phần tử (*Element-wise Broadcast*):

$$w[n] = \left(0.54 - 0.46 \cdot \cos\left(\frac{2\pi n}{399}\right) \right)^{0.85}, \quad n \in [0, 399] \quad (2)$$

$$X_{\text{win}}[t, n] = X_{\text{frames}}[t, n] \odot w[n] \quad (3)$$

- **Biến đổi Shape:** $[1, 1500, 400] \odot [400] \rightarrow [1, 1500, 400]$.

Khối 3: Discrete Fourier Transform (DFT bằng MatMul)

- **Thách thức NPU:** Thuật toán FFT (Cooley-Tukey) đòi hỏi đảo bit và truy xuất bộ nhớ không liên tục, gây nghẽn RAM.
- **Giải pháp kiến trúc:** Tách chuỗi Fourier thành 2 ma trận hằng số Thực (W_{real}) và Ảo (W_{imag}) kích thước $[400, 201]$ với $N_{\text{fft}} = 512$:

$$W_{\text{real}}[n, k] = \cos\left(\frac{2\pi nk}{512}\right), \quad W_{\text{imag}}[n, k] = -\sin\left(\frac{2\pi nk}{512}\right) \quad (4)$$

Phổ biên độ thực/ảo và phổ năng lượng (*Power Spectrum*) được tính qua MatMul:

$$S_{\text{real}} = X_{\text{win}} \cdot W_{\text{real}}, \quad S_{\text{imag}} = X_{\text{win}} \cdot W_{\text{imag}} \quad (5)$$

$$P[t, k] = (S_{\text{real}}[t, k])^2 + (S_{\text{imag}}[t, k])^2 \quad (6)$$

- **Biến đổi Shape:** $[1, 1500, 400] \times [400, 201] \rightarrow [1, 1500, 201]$.

Khối 4: Mel Filterbank Projection (Chiếu dải Mel)

- **Thách thức NPU:** Nén 201 dải tần số tuyến tính (Hertz) về 80 dải tần cảm nhận con người (Mel scale).
- **Giải pháp kiến trúc:** Khai báo ma trận bộ lọc tam giác Mel $M_{\text{mel}} \in \mathbb{R}^{201 \times 80}$ dạng **Constant Tensor** đã tính sẵn bằng Kaldi:

$$Y_{\text{mel}} = P \cdot M_{\text{mel}} \quad (7)$$

- **Biến đổi Shape:** $[1, 1500, 201] \times [201, 80] \rightarrow [1, 1500, 80]$.

Khối 5: Floor & Log Compression (Nén log dải động)

- **Thách thức NPU:** Hàm $\log(0)$ sinh ra $-\infty$ phá vỡ tính toán.
- **Giải pháp kiến trúc:** Dùng toán tử **Max** để chặn đáy năng lượng $\epsilon = 10^{-5}$ và lấy Logarithm:

$$Y_{\text{floor}}[t, m] = \max(Y_{\text{mel}}[t, m], 10^{-5}) \quad (8)$$

$$Y_{\text{fbank}}[t, m] = \ln(Y_{\text{floor}}[t, m]) \quad (9)$$

- **Biến đổi Shape:** $[1, 1500, 80] \rightarrow [1, 1500, 80]$ (Chuẩn Fbank cho Zipformer Encoder).

3. Lý do kiến trúc tối ưu cho NPU

1. **Static Allocation:** Mọi Tensor có Shape cố định ($240000 \rightarrow 1500 \times 400 \rightarrow 1500 \times 201 \rightarrow 1500 \times 80$), giúp QNN Compiler tối ưu hoá địa chỉ SRAM/VTM tuyệt đối.
2. **Loại bỏ Control Flow:** Không có câu lệnh **if/else** hay vòng lặp trượt frame.
3. **Tối ưu hoá MatMul:** 95% khối lượng tính toán rơi vào phép nhân ma trận, khai thác tối đa năng lực HTP/Tensor Cores của NPU Qualcomm.

2. Kiến trúc Acoustic Model: Zipformer Encoder & CTC Head

Khối Acoustic Model đóng vai trò nòng cốt trong việc biến đổi biểu diễn phổ tần số thời gian $\mathbf{Y}_{\text{fbank}} \in \mathbb{R}^{1 \times 1500 \times 80}$ thành chuỗi phân phối xác suất từ vựng $\mathbf{Z}_{\text{logits}} \in \mathbb{R}^{1 \times 375 \times V}$. Toàn bộ kiến trúc được đóng gói thành một Đồ thị tính toán tĩnh (Static Computational Graph), tối ưu hóa việc phân bổ bộ nhớ nội tại (VTM/SRAM) và khai thác tối đa các đơn vị tính toán VPU/HTP trên NPU Qualcomm.

Khối 2.1: Conv2D Subsampling (Giảm mẫu thời gian $4\times$)

- **Cấu trúc & Toán học:** Sử dụng 2 lớp Tích chập 2D (Conv2D) xếp chồng với bước nhảy stride = 2:

$$\mathbf{X}_0 = \text{Conv2D}_2\left(\text{Act}(\text{Conv2D}_1(\mathbf{Y}_{\text{fbank}}))\right) \quad (10)$$

- **Biến đổi Tensor:**

$$[1, 1500, 80] \xrightarrow{\text{Conv2D}_1} [1, 750, 40, C_1] \xrightarrow{\text{Conv2D}_2} [1, 375, D_{\text{model}}] \quad (11)$$

với D_{model} là chiều ẩn ban đầu (thường $D_{\text{model}} = 512$).

- **Tối ưu NPU:** Việc nén độ dài chuỗi từ $T = 1500$ xuống $T' = 375$ (mỗi khung thời gian đại diện cho 40ms) giúp giảm độ phức tạp tính toán của cơ chế Self-Attention ở các lớp sau từ $\mathcal{O}(1500^2)$ xuống $\mathcal{O}(375^2)$ (giảm $16\times$), giải phóng đáng kể tải bộ nhớ SRAM.

Khối 2.2: Zipformer Encoder Core (Trích xuất đặc trưng đa tỷ lệ)

- **Cấu trúc & Cơ chế toán học:** Zipformer cải tiến Conformer bằng cách chia Encoder thành các chặng (Stacks) hoạt động ở các độ phân giải thời gian khác nhau (Multi-rate Temporal Resolution). Mỗi khối con (*Zipformer Block*) bao gồm 3 thành phần chính:

1. **Feed-Forward Module (FFN):** Cấu trúc Macaron-style FFN với hàm kích hoạt Swish/SiLU:

$$\mathbf{X}_{\text{ffn}} = \mathbf{X} + \frac{1}{2} \cdot \text{FFN}(\mathbf{X}) \quad (12)$$

2. **Multi-Head Self-Attention (MHSA):** Trích xuất quan hệ phụ thuộc chuỗi xa (Global Context):

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (13)$$

3. **Depthwise Separable Convolution Module:** Mô hình hóa đặc trưng âm thanh cục bộ (Local Context):

$$\mathbf{X}_{\text{conv}} = \text{Pointwise}\left(\text{Depthwise}(\mathbf{X})\right) \quad (14)$$

- **Biến đổi Tensor:**

$$[1, 375, D_{\text{model}}] \xrightarrow{\text{Zipformer Stacks}} [1, 375, 768] \quad (\mathbf{H}_{\text{enc}}) \quad (15)$$

- **Tối ưu NPU:** Các lớp Depthwise Conv1D có lượng tham số nhỏ và truy cập bộ nhớ tuần tự (Sequential Memory Access), đáp ứng hoàn hảo cơ chế xử lý song song trên Vector Processing Unit (VPU) của NPU.

Khối 2.3: CTC Projection Head (`ctc_output.1`)

- **Cấu trúc & Toán học:** Sử dụng một lớp Tuyến tính (Linear Projection) chiếu biểu diễn 768 chiều sang không gian phân phối xác suất của từ vựng V ($V = 6000$ tokens):

$$\mathbf{Z}_{\text{logits}}[t, v] = \mathbf{H}_{\text{enc}}[t, :] \cdot \mathbf{W}_{\text{ctc}} + \mathbf{b}_{\text{ctc}} \quad (16)$$

trong đó $\mathbf{W}_{\text{ctc}} \in \mathbb{R}^{768 \times V}$ và $\mathbf{b}_{\text{ctc}} \in \mathbb{R}^V$.

- **Biến đổi Tensor:**

$$[1, 375, 768] \times [768, V] \longrightarrow [1, 375, V] \quad (17)$$

- **Tối ưu NPU:** Đây là phép nhân ma trận thuần túy (**MatMul / GEMM**), khai thác tối đa các lõi Ma trận (Tensor Cores / HTP) với tốc độ thực thi dưới 1ms.

Đặc tính tối ưu phần cứng trên Qualcomm HTP

1. **Static Memory Mapping:** Mọi Kích thước Tensor (Shape) từ $[1, 1500, 80] \rightarrow [1, 375, 768] \rightarrow [1, 375, V]$ đều cố định tuyệt đối, giúp QNN Compiler tính toán địa chỉ SRAM/VTM chính xác mà không tốn chi phí cấp phát bộ nhớ động (Dynamic Allocation Overhead).
2. **Tương thích Quantization (INT8/FP16):** Cấu trúc không chứa các toán tử phân nhánh (**if/else**) hay vòng lặp động, đảm bảo biên độ giá trị Tensor ổn định, tránh hiện tượng sụt giảm độ chính xác khi ép kiểu Quantization qua Qualcomm AI Hub.

3. Kiến trúc CTC Collapse: ArgMax & Lọc Trùng/Blank trên Static Graph

Khối CTC Collapse chịu trách nhiệm thu gọn chuỗi dự đoán thô $\mathbf{Z}_{\text{logits}} \in \mathbb{R}^{1 \times 375 \times V}$ thành chuỗi Token ID ngữ âm rút gọn. Do mô hình ASR dự đoán theo từng khung thời gian cố định (40ms/frame), một âm tiết kéo dài thường bị lặp lại nhiều lần liên tiếp và xen kẽ bởi ký tự trống (**<blank>**, ID = 0). Thách thức lớn nhất khi thực thi khối này trên NPU là phải loại bỏ các phần tử dư thừa nhưng vẫn duy trì ****Kích thước Đồ thị Tĩnh (Static Graph)**** mà không dùng vòng lặp động hay cấp phát bộ nhớ linh hoạt.

Khối 3.1: ArgMax & Trích xuất Token theo Khung (Frame-level Decoding)

- **Toán học:** Tại mỗi khung thời gian $t \in [1, 375]$, NPU thực hiện chọn Token ID có giá trị Logit cao nhất:

$$y_t = \arg \max_{v \in \{0, \dots, V-1\}} \mathbf{Z}_{\text{logits}}[1, t, v], \quad \forall t \in [1, 375] \quad (18)$$

- **Biến đổi Tensor:**

$$[1, 375, V] \xrightarrow{\text{ArgMax (axis=-1)}} [1, 375] \quad (\mathbf{Y} = [y_1, y_2, \dots, y_{375}]) \quad (19)$$

- **Tối ưu NPU:** Phép toán **ArgMax** trên trục từ vựng được phần cứng Vector Processing Unit (VPU) hỗ trợ song song hóa trực tiếp, cho kết quả dưới 0.1ms.

Khối 3.2: Logic Thu gọn CTC (CTC Collapse Algorithm)

- Cơ chế 2 bước:

1. **Gộp trùng (Deduplication):** Tạo Mặt nạ chỉ số (Indicator Mask) để đánh dấu các phần tử thay đổi so với phần tử liền trước:

$$m_{\text{dedup}}[t] = \mathbb{I}(y_t \neq y_{t-1}), \quad \text{với } y_0 = \text{None} \quad (20)$$

2. **Lọc Blank Token ($\epsilon = 0$):** Loại bỏ các vị trí chứa ký tự trống:

$$m_{\text{valid}}[t] = m_{\text{dedup}}[t] \wedge \mathbb{I}(y_t \neq 0) \quad (21)$$

- Ví dụ chuỗi biến đổi:

$$\begin{aligned} \mathbf{Y}_{\text{raw}} &= [0, 0, 15, 15, 15, 0, 42, 42, 0] \quad (\text{Khung ban đầu}) \\ \xrightarrow{\text{Deduplicate}} \mathbf{Y}' &= [0, 15, 0, 42, 0] \quad (\text{Loại bỏ lặp liên tiếp}) \\ \xrightarrow{\text{Remove Blank}} \mathbf{Y}_{\text{clean}} &= [15, 42] \quad (\text{Ký tự thực: "học sinh"}) \end{aligned}$$

Khối 3.3: Kỹ thuật Tĩnh hóa Đồ thị trên NPU (Static Graph Packing)

- **Thách thức NPU:** NPU Qualcomm Hexagon không hỗ trợ câu lệnh `push_back` hay mảng co giãn độ dài theo dữ liệu đầu vào.
- **Giải pháp Kiến trúc:** Sử dụng kết hợp hai toán tử `CumSum` (Prefix Sum) và `ScatterElements` để dồn các Token hợp lệ về đầu Tensor cố định:

1. **Tính vị trí đích (Destination Index):**

$$p_t = \sum_{i=1}^t m_{\text{valid}}[i] \cdot m_{\text{valid}}[t] \quad (22)$$

Toán tử `CumSum` tính toán vị trí chỉ số mới cho từng phần tử hợp lệ trong mảng.

2. **Gom phần tử (Static Scatter):** Dùng toán tử `Where` hoặc `ScatterElements` để chuyển các giá trị y_t thỏa mãn $m_{\text{valid}}[t] = 1$ về chỉ số p_t tương ứng trong Tensor đầu ra tĩnh $\mathbf{T}_{\text{out}} \in \mathbb{R}^{1 \times 375}$.
3. **Padding:** Các vị trí $k > K$ (với $K = \sum m_{\text{valid}}$ là số Token thực tế) được đệm tự động bằng số 0.

- **Biến đổi Tensor:**

$$[1, 375] \xrightarrow{\text{CumSum} + \text{Scatter}} [1, 375] \quad (\text{Chứa } K \text{ Tokens hợp lệ đầu mảng, còn lại đệm 0}) \quad (23)$$

Đặc tính tối ưu phần cứng trên Qualcomm HTP

1. **Loại bỏ hoàn toàn Host Interactivity:** Toàn bộ quá trình post-processing (lọc lặp & blank) được đóng gói 100% bên trong Đồ thị tính toán của NPU, loại bỏ chi phí truyền dữ liệu trung gian giữa NPU và CPU (Zero Data Transfer Overhead).
2. **An toàn bộ nhớ (Deterministic Memory Footprint):** Tensor đầu ra giữ nguyên Shape [1, 375] giúp QNN Compiler tối ưu hóa việc quản lý bộ nhớ đệm VTCM tĩnh, đảm bảo mô hình thực thi ổn định với độ trễ cố định (Deterministic Latency).

4. Kiến trúc Detokenize: Trích xuất Byte & Giải mã Văn bản UTF-8

Khối Detokenize đóng vai trò là công đoạn cuối cùng trong Đồ thị tính toán tĩnh End-to-End trên NPU, thực hiện chuyển đổi mảng Token ID đã được làm sạch $\mathbf{T}_{\text{out}} \in \mathbb{R}^{1 \times 375}$ thành luồng mã Byte văn bản UTF-8 đọc được. Bằng cách nhúng trực tiếp Bảng từ vựng BPE (Byte-Pair Encoding) dưới dạng ma trận hằng số tĩnh (**Constant Tensor**), NPU có thể giải mã văn bản hoàn toàn nội tại mà không cần gọi các thư viện ngoài (như SentencePiece hay HuggingFace Tokenizers) trên CPU Host.

Khối 4.1: Bảng tra cứu Byte tĩnh (Static Byte Lookup Table)

- **Cấu trúc & Khởi tạo Offline:** Xây dựng ma trận ma phác thảo các chuỗi Byte UTF-8 $M_{\text{byte}} \in \mathbb{R}^{V \times L_{\text{max}}}$ từ file từ vựng `bpe.model`:
 - V : Kích thước từ điển ($V = 6000$ tokens).
 - L_{max} : Độ dài byte tối đa cho một Token (thường chọn $L_{\text{max}} = 16$ bytes).

Mỗi hàng $v \in [0, V - 1]$ trong M_{byte} lưu trữ chuỗi mã ASCII/UTF-8 tương ứng của token thứ v , các vị trí còn thiếu được đệm tự động bằng số 0 (**NULL byte**).

- **Ví dụ Bảng tra cứu:**

$$M_{\text{byte}}[15, :] = [32, 104, 225, 120, 185, 0, \dots, 0] \quad (\text{Biểu diễn UTF-8 của " học"}) \quad (24)$$

$$M_{\text{byte}}[42, :] = [32, 115, 105, 110, 104, 0, \dots, 0] \quad (\text{Biểu diễn UTF-8 của " sinh"}) \quad (25)$$

Khối 4.2: Phép tra cứu Gather & Ghép chuỗi Byte (Static Byte Gathering & Flattening)

- **Toán học:** NPU thực hiện toán tử **Gather** (Index Select) trực tiếp trên trục đầu tiên của M_{byte} theo danh sách Token ID:

$$\mathbf{B}_{\text{tokens}} = \text{Gather}(M_{\text{byte}}, \mathbf{T}_{\text{out}}, \text{axis} = 0) \quad (26)$$

- **Duỗi phẳng mảng Byte (Flattening):** Biến đổi ma trận 2D thành luồng 1D tính để thu được chuỗi Byte văn bản hoàn chỉnh:

$$\mathbf{B}_{\text{stream}} = \text{Reshape}(\mathbf{B}_{\text{tokens}}, [1, 375 \times L_{\text{max}}]) \quad (27)$$

- **Biến đổi Tensor:**

$$[1, 375] \xrightarrow{\text{Gather}(M_{\text{byte}})} [1, 375, 16] \xrightarrow{\text{Reshape}} [1, 6000] \quad (\text{Mảng Byte tĩnh}) \quad (28)$$

- **Chuỗi Byte đầu ra thực tế:**

$$\mathbf{B}_{\text{stream}} = [32, 104, 225, 120, 185, 32, 115, 105, 110, 104, 0, 0, \dots] \implies \text{“học sinh”} \quad (29)$$

Khối 4.3: Xuất dữ liệu ra Host CPU (Zero-CPU Decoding)

- **Cơ chế xử lý tại Host:** Ứng dụng ở tầng Host (C++/Python/Android NDK) chỉ cần nhận mảng Byte thô $[1, 6000]$ từ NPU và thực hiện ép kiểu string trực tiếp:

$$\text{Text} = \text{StringDecode}(\mathbf{B}_{\text{stream}}, \text{encoding} = \text{'utf-8'}, \text{trim} = \text{'\0'}) \quad (30)$$

- Không tồn bất kỳ chi phí tính toán Tokenizer nào trên CPU.

Đặc tính tối ưu phần cứng trên Qualcomm HTP

1. **End-to-End Static DAG hoàn chỉnh:** Tích hợp thành công toàn bộ Pipeline ASR (Phổ Mel $\rightarrow$ Encoder $\rightarrow$ CTC Collapse $\rightarrow$ UTF-8 Bytes) thành 1 tập tin mô hình tĩnh duy nhất (`.onnx` / `.serialized.bin`).
2. **Tối ưu hóa bộ nhớ VTCM via Gather:** Toán tử **Gather** tra cứu dữ liệu từ bảng M_{byte} nằm trực tiếp trên bộ nhớ đệm VTCM (Vector Tightly-Coupled Memory) của Hexagon NPU, giúp truy xuất dữ liệu với độ trễ gần như bằng 0.
3. **Loại bỏ điểm nghẽn Bus (Data Transfer Bottleneck):** Việc chuyển đổi trực tiếp sang chuỗi Byte trên NPU giúp kích thước dữ liệu truyền ngược về CPU đạt mức tối thiểu, loại bỏ hiện tượng nghẽn băng thông Bus PCIe/AXI.